

LAB 006

INFRASTRUCTURE AS CODE WITH TERRAFORM

[Build Your Entire AWS Infrastructure with Code](#)

VPC • Subnets • NAT Gateway • EC2 • ALB • Auto Scaling • Secrets Manager

Everything you built by hand in previous labs — now automated with one command.

Part 1: What is Infrastructure as Code?

1.1 The Problem

In your previous labs, you created VPCs, subnets, security groups, EC2 instances, load balancers, and more, all by clicking through the AWS Console. This works for learning, but it has serious problems in the real world:

- **Not repeatable:** If you need to create the same environment again (for staging, for a new region, for disaster recovery), you have to click through everything again. Did you remember every setting?
- **Not version-controlled:** There's no Git history for clicks. You can't see who changed what, when, or why.
- **Error-prone:** Miss one checkbox, typo one CIDR block, forget one security group rule, and things break silently.
- **Slow:** Creating 20+ resources by hand takes an hour. Doing it with code takes seconds.
- **Not shareable:** You can't hand someone a screenshot and say "build this." You can hand them a Terraform file.

1.2 The Solution: Infrastructure as Code (IaC)

Infrastructure as Code means you define your infrastructure (servers, networks, databases, load balancers) in text files. You check those files into Git. A tool reads those files and creates the actual resources in the cloud. If you want to change something, you edit the file and re-apply. If you want to destroy everything, one command tears it all down.

1.3 What is Terraform?

Terraform is an open-source IaC tool created by HashiCorp. It is the most widely used IaC tool in the industry. Unlike AWS CloudFormation (which only works with AWS), Terraform works with AWS, Azure, GCP, and hundreds of other providers.

Concept	Explanation
Provider	A plugin that lets Terraform talk to a specific platform (e.g., aws, azure, google). You configure it with your credentials.
Resource	A single infrastructure object: a VPC, a subnet, an EC2 instance, a security group. Each resource is defined in a resource block.

Data Source	A way to read information from AWS without creating anything. Example: look up the latest Amazon Linux AMI ID.
Variable	An input parameter that makes your code reusable. Instead of hardcoding us-east-1, you use var.region.
Output	A value displayed after terraform apply. Example: the ALB DNS name, the bastion public IP.
State File	terraform.tfstate: a JSON file that tracks what Terraform has created. This is how Terraform knows what exists and what needs to change.
Module	A reusable package of Terraform code. Like a function in programming. (We won't use modules in this lab.)

1.4 The Terraform Lifecycle

Terraform has three main commands you'll use constantly:

```

terraform init      # Download provider plugins. Run once per project.
terraform plan      # Preview what will be created/changed/destroyed.
terraform apply     # Actually create/change the resources.
terraform destroy   # Tear down everything Terraform created.
```

Command	What It Does
terraform init	Downloads the AWS provider plugin. Creates .terraform/ directory. Run this first in any new project.
terraform plan	Reads your .tf files, compares them to the state file, and shows you what it WILL do. Nothing is created. This is your safety net.
terraform apply	Executes the plan. Creates/updates/deletes resources in AWS. Updates the state file. You must type "yes" to confirm.
terraform destroy	Destroys ALL resources that Terraform created. Read the state file to know what to delete. You must type "yes" to confirm.

Note

Always run "terraform plan" before terraform apply. Read the plan carefully. If it says it will destroy something you don't expect, stop and investigate. The plan is your safety net.

1.5 — HCL (HashiCorp Configuration Language)

Terraform uses its own language called HCL. It looks like this:

```
# This is a comment
resource "aws_vpc" "main" {      # resource TYPE and NAME
  cidr_block = "10.0.0.0/16"    # argument = value

  tags = {                      # map of key-value pairs
    Name = "Lab-VPC"
  }
}
```

- **resource** is the keyword for creating something
- **"aws_vpc"** is the resource type (AWS provider's VPC resource)
- **"main"** is your local name for this resource (used to reference it elsewhere)
- **cidr_block** is an argument specific to the aws_vpc resource
- **tags** is a map (dictionary) that adds name tags visible in the console

To reference this VPC elsewhere, you use: `aws_vpc.main.id`

Part 2: Setup: Install Terraform & Configure AWS

2.1 Install Terraform

macOS (Homebrew)

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
terraform -version
```

Windows

Download from <https://developer.hashicorp.com/terraform/downloads> and add to PATH.

2.2 Configure AWS Credentials

Terraform needs AWS access keys to create resources. If you don't have the AWS CLI installed, install it first from <https://aws.amazon.com/cli/>

Details here:

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Step 1: Create an Access Key in AWS

- 1- Go to the AWS Console → IAM → Users → select your user.
- 2- Click the "Security credentials" tab.
- 3- Under "Access keys", click "Create access key".
- 4- Select "Command Line Interface (CLI)" as the use case.
- 5- Download the CSV or copy the Access Key ID and Secret Access Key.

Step 2: Configure the AWS CLI

- 1- Open your terminal and run:

```
aws configure
```

- 2- Enter the following when prompted:

AWS Access Key ID: <your-access-key-id>

AWS Secret Access Key: <your-secret-access-key>

Default region name: `us-east-1` (or your preferred region)

Default output format: `json`

⚠ Warning

Never commit your AWS credentials to Git. Never put them in `.tf` files. Terraform automatically reads them from the AWS CLI configuration (`~/.aws/credentials`). This is the safest approach.

Step 3: Verify the Connection

Run a quick test to confirm Terraform can reach AWS:

```
aws sts get-caller-identity
```

```
mohammad@Mohammads-Laptop ~ % aws sts get-caller-identity
{
  "UserId": "AIDA5I64234ICXPUSYUIW",
  "Account": "912606813968",
  "Arn": "arn:aws:iam::912606813968:user/admin"
}
```

You should see your AWS account ID and user ARN. If this works, Terraform will work.

Note: In case you have previously did aws configure then you might need to do:

```
rm ~/.aws/credentials
```

```
rm ~/.aws/config
```

2.3 Create the Project Directory

Step 4: Set Up the Terraform Project

Create a folder for this lab and the initial file structure:

```
mkdir terraform-lab && cd terraform-lab
touch main.tf network.tf security.tf compute.tf alb.tf asg.tf secrets.tf
variables.tf outputs.tf
mkdir userdata
```

Your folder structure should now look like this:

```
terraform-lab/
├── main.tf           # Provider + VPC
├── network.tf        # Subnets, IGW, NAT GW, route tables
├── security.tf       # Security groups
├── compute.tf        # EC2 instances
├── alb.tf            # Load balancer
├── asg.tf            # Auto Scaling
├── secrets.tf        # Parameter Store secrets
├── variables.tf      # Input variables
├── outputs.tf        # Output values
└── userdata/        # Bootstrap scripts
```

**Tip**

Terraform reads ALL .tf files in the current directory and merges them. The file names don't matter to Terraform, we split them for organization and readability.

Part 3: Provider & VPC (main.tf)

Let's start writing Terraform code. We begin with the provider configuration and the VPC, the foundation of everything.

Step 5: Configure the AWS Provider (main.tf)

Open main.tf in your code editor and add:

```
# — Terraform Configuration —
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# — AWS Provider —
provider "aws" {
  region = var.aws_region
}
```

This tells Terraform: use the AWS provider (version 5.x), and deploy to the region defined in our variables.

Step 6: Define the Variables We Need (variables.tf)

Open variables.tf and add our first variables:

```
variable "aws_region" {
  description = "AWS region to deploy resources"
  type        = string
  default     = "us-east-1"
}

variable "vpc_cidr" {
  description = "CIDR block for the VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnet_cidr" {
  description = "CIDR block for the public subnet"
  type        = string
  default     = "10.0.1.0/24"
}
```



```

variable "private_subnet_cidr" {
  description = "CIDR block for the private subnet"
  type        = string
  default     = "10.0.2.0/24"
}

variable "public_subnet_cidr_2" {
  description = "CIDR block for second public subnet (ALB needs 2 AZs)"
  type        = string
  default     = "10.0.3.0/24"
}

variable "key_name" {
  description = "Name of the SSH key pair"
  type        = string
}

variable "my_ip" {
  description = "Your public IP for SSH access (e.g., 203.0.113.50/32)"
  type        = string
}

```

**Note**

Variables without a default (like `key_name` and `my_ip`) will prompt you for a value when you run terraform apply.

You can also create a `terraform.tfvars` file to set them automatically (we'll do this later).

Step 7: Create the VPC (main.tf)

Add the VPC resource below the provider in `main.tf`:

```

# — VPC —
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Lab-VPC"
  }
}

```

This is the Terraform equivalent of creating a VPC in the console. Compare:

Console: VPC Dashboard → Create VPC → fill in CIDR, check DNS hostnames → Click Create

Terraform: 6 lines of code. Repeatable. Version-controlled.

Step 8: Initialize Terraform

Now let's initialize Terraform to download the AWS provider:

```
terraform init
```

You should see: "Terraform has been successfully initialized!"

This created a `.terraform/` directory with the AWS provider plugin.

 Tip

Run `terraform init` whenever you add a new provider or change provider versions. You only need to run it once per project, unless the provider config changes.

Part 4: Networking (network.tf)

Now we create all the networking components: subnets, Internet Gateway, NAT Gateway, and route tables. Open `network.tf` and add each block:

4.1 Data Source: Availability Zones

Instead of hardcoding availability zones, we'll let Terraform look them up:

Step 9: Add AZ Data Source (network.tf)

```
# — Availability Zones —  
data "aws_availability_zones" "available" {  
  state = "available"  
}
```

This data source fetches the list of available AZs in your region. We'll reference it as `data.aws_availability_zones.available.names[0]` for the first AZ.

4.2 Subnets

Step 10: Create Subnets (network.tf)

```
# — Public Subnet 1 —
resource "aws_subnet" "public_1" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet_cidr
  availability_zone  = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true

  tags = { Name = "Lab-Public-Subnet-1" }
}

# — Public Subnet 2 (needed for ALB - must be in different AZ) —
resource "aws_subnet" "public_2" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet_cidr_2
  availability_zone  = data.aws_availability_zones.available.names[1]
  map_public_ip_on_launch = true

  tags = { Name = "Lab-Public-Subnet-2" }
}

# — Private Subnet —
resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet_cidr
  availability_zone  = data.aws_availability_zones.available.names[0]

  tags = { Name = "Lab-Private-Subnet" }
}
```

Notice how we reference the VPC: `aws_vpc.main.id` — Terraform automatically knows to create the VPC first, then the subnets. This is called an implicit dependency.



Note

We need 2 public subnets in different AZs because ALB requires subnets in at least 2 Availability Zones. This is the same requirement you encountered in Lab 003.

4.3 Internet Gateway

Step 11: Create and Attach IGW (network.tf)

```
# — Internet Gateway —
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
}
```

```
tags = { Name = "Lab-IGW" }
}
```

In the console, you had to create the IGW and then separately attach it. In Terraform, the `vpc_id` argument does both in one step.

4.4 NAT Gateway

Step 12: Create Elastic IP and NAT Gateway (network.tf)

```
# — Elastic IP for NAT Gateway —
resource "aws_eip" "nat" {
  domain = "vpc"

  tags = { Name = "Lab-NAT-EIP" }
}

# — NAT Gateway (in public subnet) —
resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public_1.id

  tags = { Name = "Lab-NAT-GW" }

  depends_on = [aws_internet_gateway.main]
}
```

The `depends_on` is an explicit dependency. We tell Terraform: don't create the NAT Gateway until the Internet Gateway exists. This is needed because the NAT Gateway routes through the IGW.

4.5 Route Tables

Step 13: Create Route Tables and Associations (network.tf)

```
# — Public Route Table —
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = { Name = "Lab-Public-RT" }
}
```

```

}

# — Associate public subnets with public route table —
resource "aws_route_table_association" "public_1" {
  subnet_id      = aws_subnet.public_1.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_2" {
  subnet_id      = aws_subnet.public_2.id
  route_table_id = aws_route_table.public.id
}

# — Private Route Table —
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }

  tags = { Name = "Lab-Private-RT" }
}

resource "aws_route_table_association" "private" {
  subnet_id      = aws_subnet.private.id
  route_table_id = aws_route_table.private.id
}

```

Compare this to the VPC lab: in the console you had to create the route table, add routes, then associate subnets — 3 separate steps. In Terraform, it's one cohesive block.

Step 14: Verify Your Progress — First Plan!

Let's run our first terraform plan to check for errors:

```
terraform plan -var="key_name=your-key" -var="my_ip=YOUR.IP.HERE/32"
```

You should see something like: "Plan: 12 to add, 0 to change, 0 to destroy."

If you see errors, read the error message carefully and fix the .tf files.

Use 0.0.0.0/0 as your IP, And your-key, replace it with the key you created before (ex: my-ec2.pem)

**Tip**

Don't terraform apply yet! We'll add all the resources first, then deploy everything at once.

Part 5: Security Groups (security.tf)

Security groups in Terraform work the same way as in the console, but they're much cleaner. We can reference other security groups by their Terraform resource name.

Step 15: Create All Security Groups (security.tf)

```
# — Bastion Security Group —
resource "aws_security_group" "bastion" {
  name          = "Bastion-SG"
  description   = "Allow SSH from my IP"
  vpc_id        = aws_vpc.main.id

  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "Bastion-SG" }
}

# — ALB Security Group —
resource "aws_security_group" "alb" {
  name          = "ALB-SG"
  description   = "Allow HTTP from internet"
  vpc_id        = aws_vpc.main.id

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```

    egress {
      from_port    = 0
      to_port      = 0
      protocol     = "-1"
      cidr_blocks = ["0.0.0.0/0"]
    }

    tags = { Name = "ALB-SG" }
  }

# — Web Server Security Group —
resource "aws_security_group" "web" {
  name          = "Web-SG"
  description   = "Allow HTTP from ALB, SSH from bastion"
  vpc_id        = aws_vpc.main.id

  ingress {
    description = "HTTP from ALB"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  ingress {
    description = "SSH from bastion"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    security_groups = [aws_security_group.bastion.id]
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "Web-SG" }
}

```

Notice the `security_groups = [aws_security_group.alb.id]` — this is the Terraform equivalent of referencing another security group as the source in the console. Exactly what you learned in Lab 002/2!



Note

The egress block with `protocol = "-1"` and `0.0.0.0/0` means "allow all outbound traffic" — the same as the default outbound rule in the console.

Part 6: Compute: EC2 Instances (compute.tf)

6.1 Find the Latest AMI Automatically

Instead of hardcoding an AMI ID (which changes per region), we use a data source:

Step 16: Add AMI Data Source (compute.tf)

```
# — Latest Amazon Linux 2023 AMI —
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

This automatically finds the latest Amazon Linux 2023 AMI. No more looking it up in the console! Reference it as `data.aws_ami.amazon_linux.id`

6.2 Bastion Host

Step 17: Create Bastion Host (compute.tf)

```
# — Bastion Host —
resource "aws_instance" "bastion" {
  ami              = data.aws_ami.amazon_linux.id
  instance_type    = "t2.micro"
  subnet_id        = aws_subnet.public_1.id
  vpc_security_group_ids = [aws_security_group.bastion.id]
  key_name          = var.key_name

  tags = { Name = "Bastion-Host" }
}
```


6.3 Web Server User Data Script

Step 18: Create Bootstrap Script (userdata/web.sh)

Create the file `userdata/web.sh` with the following content:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd

# Get instance metadata (IMDSv2)
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/placement/availability-zone)

# Create web page
cat > /var/www/html/index.html <<EOF
<h1>Hello from Terraform!</h1>
<p>Instance ID: $INSTANCE_ID</p>
<p>Availability Zone: $AZ</p>
<p>Deployed with Infrastructure as Code</p>
EOF
```

This is a similar user data pattern from Lab 003, but now it's in a separate file, version-controlled, and reusable.

6.4 Web Server EC2 (for initial testing)

Step 19: Create a Web Server Instance (compute.tf)

```
# — Web Server (for initial testing before ASG) —
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.public_1.id
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name      = var.key_name
  user_data     = file("userdata/web.sh")

  tags = { Name = "Web-Server" }
}
```

The `file()` function reads the user data script from disk. Clean and maintainable.

Part 7: Application Load Balancer (alb.tf)

The ALB configuration in Terraform maps directly to what you did in Lab 003: create an ALB, a target group, and a listener.

Step 20: Create ALB, Target Group, and Listener (alb.tf)

```
# — Application Load Balancer —
resource "aws_lb" "web" {
  name           = "Lab-ALB"
  internal       = false
  load_balancer_type = "application"
  security_groups = [aws_security_group.alb.id]
  subnets       = [aws_subnet.public_1.id, aws_subnet.public_2.id]

  tags = { Name = "Lab-ALB" }
}

# — Target Group —
resource "aws_lb_target_group" "web" {
  name      = "Lab-TG"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = aws_vpc.main.id

  health_check {
    path           = "/"
    protocol       = "HTTP"
    healthy_threshold = 2
    unhealthy_threshold = 2
    timeout        = 5
    interval       = 10
  }

  tags = { Name = "Lab-TG" }
}

# — Register the web server with the target group —
resource "aws_lb_target_group_attachment" "web" {
  target_group_arn = aws_lb_target_group.web.arn
  target_id        = aws_instance.web.id
  port             = 80
}

# — Listener —
resource "aws_lb_listener" "web" {
  load_balancer_arn = aws_lb.web.arn
  port              = 80
  protocol          = "HTTP"
}
```

```

default_action {
  type          = "forward"
  target_group_arn = aws_lb_target_group.web.arn
}
}

```

Part 8: Auto Scaling Group (asg.tf)

Auto Scaling in Terraform uses a Launch Template and an ASG — the same concepts from Lab 003/1 and 003/2.

Step 21: Create Launch Template and ASG (asg.tf)

```

# — Launch Template —
resource "aws_launch_template" "web" {
  name_prefix    = "Lab-LT-"
  image_id       = data.aws_ami.amazon_linux.id
  instance_type  = "t2.micro"
  key_name       = var.key_name

  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = base64encode(file("userdata/web.sh"))

  tag_specifications {
    resource_type = "instance"
    tags = { Name = "ASG-Web-Server" }
  }
}

# — Auto Scaling Group —
resource "aws_autoscaling_group" "web" {
  name                = "Lab-ASG"
  desired_capacity     = 2
  min_size            = 1
  max_size            = 4
  vpc_zone_identifier = [aws_subnet.public_1.id, aws_subnet.public_2.id]
  target_group_arns   = [aws_lb_target_group.web.arn]

  launch_template {
    id       = aws_launch_template.web.id
    version = "$Latest"
  }
}

```

```

    }

    health_check_type      = "ELB"
    health_check_grace_period = 120

    tag {
      key          = "Name"
      value        = "ASG-Web-Server"
      propagate_at_launch = true
    }
  }

  # — Scaling Policy (scale up when CPU > 70%) —
  resource "aws_autoscaling_policy" "scale_up" {
    name                = "Lab-Scale-Up"
    autoscaling_group_name = aws_autoscaling_group.web.name
    policy_type          = "TargetTrackingScaling"

    target_tracking_configuration {
      predefined_metric_specification {
        predefined_metric_type = "ASGAverageCPUUtilization"
      }
      target_value = 70.0
    }
  }
}

```

Notice `target_group_arns` in the ASG, this is the critical link between the ASG and ALB (Lab 003/2). New instances automatically register with the target group.

Note

The `user_data` in a launch template must be base64-encoded, hence `base64encode(file(...))`. For a regular `aws_instance`, Terraform handles encoding automatically with the `file()` function.

Part 9: Secrets Management (secrets.tf)

9.1 Why Secrets Management?

In the real world, your applications need passwords, API keys, and database credentials. You should NEVER hardcode these in your Terraform files, user data scripts, or application code. Instead, store them in a secure service and fetch them at runtime.

9.2 AWS Secrets Manager vs. Parameter Store

Feature	Secrets Manager	Parameter Store (SSM)
Cost	\$0.40/secret/month + API calls	Free for standard parameters
Auto-Rotation	Built-in (e.g., rotate DB passwords every 30 days)	Not built-in (you script it yourself)
Encryption	Always encrypted with KMS	Optional (SecureString type)
Best For	Production: DB credentials, API keys with rotation	Learning, config values, simple secrets
Max Size	64 KB	4 KB (standard) / 8 KB (advanced)

In this lab, we'll use **SSM Parameter Store** because it's free and simpler. The concept is the same: store the secret in AWS, reference it in Terraform, and fetch it in your EC2 instances at boot time.

9.3 Create a Secret in Parameter Store

Step 22: Store a Database Password in Parameter Store (secrets.tf)

```
# — Store a secret in SSM Parameter Store —
resource "aws_ssm_parameter" "db_password" {
  name          = "/lab/database/password"
  description   = "Database password for the lab application"
  type          = "SecureString"
  value         = "MySecureP@ssw0rd123"

  tags = { Name = "Lab-DB-Password" }
}

# — Store the database host —
```

```
resource "aws_ssm_parameter" "db_host" {
  name          = "/lab/database/host"
  description   = "Database host"
  type          = "String"
  value         = "10.0.2.50"

  tags = { Name = "Lab-DB-Host" }
}
```

The `/lab/database/` path creates a hierarchy — like folders for your secrets. `SecureString` encrypts the value with AWS KMS. `String` stores it in plain text (fine for non-sensitive config).

⚠ Warning

In this lab, we hardcode the password value for simplicity. In production, you would NEVER do this.

Instead, you'd generate a random password with the `random_password` Terraform resource or set it manually in the console.

9.4 Read Secrets from EC2 at Boot Time

To actually USE the secret in an EC2 instance, the instance needs an IAM role with permission to read from Parameter Store. Here's how to add that:

Step 23: Create IAM Role for EC2 to Read Secrets (secrets.tf)

```
# — IAM Role for EC2 —
resource "aws_iam_role" "ec2_ssm" {
  name = "Lab-EC2-SSM-Role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}

# — Policy: Allow reading SSM parameters —
resource "aws_iam_role_policy" "ssm_read" {
  name = "SSM-Read-Policy"
  role = aws_iam_role.ec2_ssm.id

  policy = jsonencode({
```

```

Version = "2012-10-17"
Statement = [{
  Effect    = "Allow"
  Action    = [
    "ssm:GetParameter",
    "ssm:GetParameters",
    "ssm:GetParametersByPath"
  ]
  Resource = "arn:aws:ssm:${var.aws_region}::parameter/lab/*"
}]
})
}

# — Instance Profile (connects IAM role to EC2) —
resource "aws_iam_instance_profile" "ec2_ssm" {
  name = "Lab-EC2-SSM-Profile"
  role = aws_iam_role.ec2_ssm.name
}

```

Then add `iam_instance_profile = aws_iam_instance_profile.ec2_ssm.name` to your EC2 instances and launch template to give them access.

✓ Tip

To read the secret from inside an EC2 instance, use the AWS CLI:

```
aws ssm get-parameter --name /lab/database/password --with-decryption --region us-east-1
--query Parameter.Value --output text
```

Part 10: Outputs (outputs.tf)

Outputs display useful information after terraform apply. Open `outputs.tf` and add:

Step 24: Define Outputs (outputs.tf)

```

output "vpc_id" {
  description = "ID of the VPC"
  value      = aws_vpc.main.id
}

output "alb_dns_name" {
  description = "DNS name of the Application Load Balancer"
  value      = aws_lb.web.dns_name
}

```

```
output "bastion_public_ip" {
  description = "Public IP of the bastion host"
  value       = aws_instance.bastion.public_ip
}

output "web_server_public_ip" {
  description = "Public IP of the web server"
  value       = aws_instance.web.public_ip
}

output "db_password_ssm_path" {
  description = "SSM path for the database password"
  value       = aws_ssm_parameter.db_password.name
}
```

After terraform apply, these values will be displayed. You can also run terraform output at any time to see them again.

Part 11: Deploy & Test

11.1 Create a tfvars File

Step 25: Create terraform.tfvars

Create a file called terraform.tfvars to set your variable values:

```
aws_region = "us-east-1"
key_name    = "your-key-pair-name"
my_ip       = "YOUR.PUBLIC.IP/32"
```

To find your public IP, run: `curl -s https://checkip.amazonaws.com`

Warning

Add terraform.tfvars to your .gitignore! It may contain sensitive values. Also add: .terraform/, *.tfstate, *.tfstate.backup

11.2 Plan

Step 26: Run Terraform Plan

```
terraform plan
```

Read the output carefully. You should see 25–30 resources to be created. Verify:

- 1 VPC
- 3 subnets (2 public, 1 private)
- 1 IGW, 1 NAT GW, 1 EIP
- 2 route tables + 3 associations
- 3 security groups
- 2 EC2 instances (bastion + web)
- 1 ALB + 1 target group + 1 listener + 1 attachment
- 1 launch template + 1 ASG + 1 scaling policy
- 2 SSM parameters + IAM role + policy + instance profile

11.3 Apply

Step 27: Run Terraform Apply

```
terraform apply
```

Terraform will show the plan and ask: "Do you want to perform these actions?"

Type "yes" and press Enter.

This will take 3–5 minutes. Watch the resources being created in real time.

When complete, you'll see your outputs (ALB DNS, bastion IP, etc.).



11.4 Test

Step 28: Verify Everything Works

1. Copy the alb_dns_name from the output.
2. Open it in your browser: `http://<ALB-DNS-NAME>`
3. You should see the "Hello from Terraform!" page with the instance ID.
4. Refresh several times — the instance ID should change (ASG has 2 instances).
5. SSH into the bastion to verify access:

```
ssh -i your-key.pem ec2-user@<BASTION-PUBLIC-IP>
```

6. Verify the SSM parameter from the bastion:

```
aws ssm get-parameter --name /lab/database/password \
  --with-decryption --region us-east-1 \
  --query Parameter.Value --output text
```

Congratulations! You just created an entire AWS infrastructure: VPC, subnets, NAT Gateway, security groups, EC2 instances, ALB, Auto Scaling, and secrets, all with code. One command up, one command down.

Part 12: Clean Up, Destroy Everything

Step 29: Destroy All Resources

```
terraform destroy
```

- 1- Terraform will show every resource it will delete and ask for confirmation.
- 2- Type "yes" and press Enter.
- 3- Wait for all resources to be destroyed.
- 4- Verify in the AWS Console that everything is gone.

✓ Tip

This is the power of Terraform. In the VPC lab, cleanup was 8 manual steps in a specific order (instances → NAT GW → EIP → SGs → subnets → route tables → IGW → VPC). Terraform handles the dependency order automatically.

12.1 The Magic Test: Recreate from Scratch

Step 30: Prove It's Fully Reproducible

Now run:

```
terraform apply
```

Type "yes" — and watch your entire infrastructure come back from nothing. This is the real value of IaC: your infrastructure is reproducible, sharable, and version-controlled.

When you're done testing, run terraform destroy one final time to avoid charges.

Lab Recap

What You Did	Terraform Resource
Created a VPC	aws_vpc
Created subnets	aws_subnet (x3)
Attached Internet Gateway	aws_internet_gateway
Created NAT Gateway + EIP	aws_nat_gateway + aws_eip
Configured route tables	aws_route_table + aws_route_table_association
Set up security groups	aws_security_group (x3)

Launched EC2 instances	aws_instance (bastion + web)
Set up ALB + target group	aws_lb + aws_lb_target_group + aws_lb_listener
Created Auto Scaling	aws_launch_template + aws_autoscaling_group
Stored secrets securely	aws_ssm_parameter + IAM role

Key Takeaways

- Infrastructure as Code means your infrastructure is repeatable, version-controlled, and shareable.
- terraform plan is your safety net — always review before applying.
- terraform destroy + terraform apply proves your infrastructure is fully reproducible.
- Use variables for anything that could change; use outputs to display useful info.
- Never hardcode secrets. Use SSM Parameter Store (free) or Secrets Manager (production).
- Terraform resolves dependencies automatically — it knows to create the VPC before the subnets.
- Split your code into multiple .tf files for organization (Terraform merges them all).

End of Lab 006 — Welcome to Infrastructure as Code!